

Solutions Engineer: Technical Assignment (HyperExecute)

Timeline: Please complete and submit within 1-2 days of receiving this.

Goal: This assignment checks your ability to read documentation, debug configs independently, and reason about test orchestration, not to test memorized syntax. Google, docs, and trial-and-error are all fair game.

Before you start

- Sign up for a LambdaTest account (free trial is fine) and get your Username + Access Key from the LambdaTest profile page.
- Read through the HyperExecute getting-started docs: <https://www.lambdatest.com/support/docs/getting-started-with-hyperexecute/>
- Clone the sample project: <https://github.com/LambdaTest/testng-selenium-hyperexecute-sample>
- Install the HyperExecute CLI as described in the docs, and confirm you can trigger a basic job before moving to the tasks below.

Task 1: Fix the broken YAML

The broken HyperExecute YAML file is here: <https://gist.github.com/vishnukdas/20f629efd5747cfbea844996ccc4ec658>. Identify every issue, fix it, and get a job running successfully against the sample TestNG project.

Deliverables:

- The corrected YAML file
- A short note (bullet points are fine) listing each error you found and why it was breaking the job
- A screenshot or link showing the job running successfully on the LambdaTest dashboard

Task 2: Environment variables

Extend the fixed YAML from Task 1 to:

- Define at least one custom environment variable in the YAML (e.g., ENVIRONMENT: staging).
- Print that environment variable's value in the pre steps (i.e., before tests run), so it's visible in the HyperExecute job logs.
- Access the same environment variable from inside a test case in the sample project and print its value to the console during test execution.

Deliverables:

- Updated YAML with the env variable defined
- The code snippet showing where/how you read the env variable in the test case
- Log output (screenshot or copy-paste) showing the value printed both in pre-steps and during test execution

Task 3: Force a failure and configure retries

- Modify or add a test case in the sample project that explicitly fails (e.g., a hard assertion failure, not a flaky/random one).
- Configure retryOnFailure in the YAML so that this failing test is retried automatically.
- Run the job and confirm from the logs/dashboard that the retry actually happened.

Deliverables:

- The test code shows the intentional failure
- The relevant YAML section for retries
- Evidence (log/dashboard screenshot) that the retry executed

Task 4: Linux/Unix basics

These are meant to be quick: they check baseline comfort with the shell, since a lot of real debugging on this role happens by grepping and parsing logs.

Using any log file or text file you like (you can use HyperExecute job logs, or generate a sample text file):

- Use grep to find all lines containing the word FAIL (or ERROR) in a log file.
- Use awk to extract and print just the second column from a space-delimited file.
- Use sed to do a find-and-replace on a string in a file (e.g., replace all occurrences of staging with production).
- Chain at least two of the above commands together with a pipe (|) to do something meaningful, e.g., filter lines with grep, then extract a field with awk.

Deliverables:

- The exact commands you used
- A one-line explanation of what each command does
- Sample input and output (doesn't need to be from a real HyperExecute log; any text file is fine)

Submission format

Please share:

- A short doc (or README) with your notes for each task
- Your corrected/modified YAML file(s) and any code changes in a GitHub repo.
- Screenshots/links as evidence for each task

We'll walk through your submission together in the next interview round and go deeper into HyperExecute's architecture, features, and use cases, so make sure you understand why each fix worked, not just that it worked.

Good luck!